

Kapitel 13. Kompilera C++-program

Innehållsförteckning

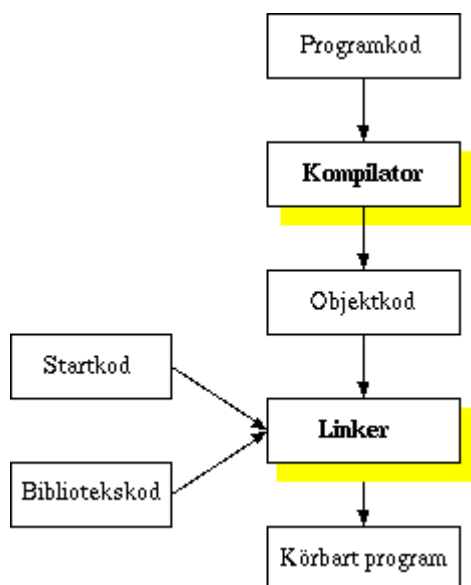
[Kompilatorer](#)[Länkning av program](#)[Använda multipla filer](#)

Detta kapitel redogör för hur program kompileras under Unix, eftersom kompilatorer finns fritt tillgängliga till de flesta versioner av Unix. Kompilering under Windows är beroende av om en programmeringsomgivning finns tillgänglig. Alla de olika omgivningarna är olika. Användaren refereras till de medföljande manualerna.

Kompilatorer

C++-kompilatorer under Unix är relativt enkla program. Deras enda uppgift är att läsa en programfil där ett C++-program finns och omvandla denna text till ett körbart program. Så gott som alla kompilatorer är *kommandoradskompilatorer*, d.v.s. användaren ger ett kommando i kommandotolken som startar kompileringen. De uppgifter som kompilatorn gör "bakom kulisserna" kan enkelt visas av följande bild:

Figur 13-1. Kompilatorns arbetsskeden



Den delen som användaren närmast stöter på är den första, själva kompileringsskedet. I detta skede körs preprocessorn och därefter försöker kompilatorn läsa den fil som skall kompileras och "förstå" den. Men detta avses att kompilatorn parsar koden i filen och bygger upp interna strukturer för hur programmet skall se ut. Om allt verkar vara korrekt och inga syntaxfel finns skapas en fil med *objektkod*. Denna objektkod är programmet assemblerform. Mera info om preprocessorn finns i [Kapitel 10](#). Därefter startas *linkern* som har som uppgift att linka in olika bibliotek, nödvändig extra kod som krävs för att starta programmet samt skriver ut det färdiga körbara programmet. Fel som kan förekomma i detta stadium är t.ex. funktioner som inte finns definierade någonstans eller bibliotek som inte kan hittas på disken. För mera information om bibliotek se [Kapitel 11](#). Vi kommer här att se

på hela kompileringsprocessen som en svart låda som antingen kompilerar vårt program eller så ger ett felmeddelande.

Kompilera program

Kompilatorer på Unix har i vissa fall olika namn, men alla fungerar i princip på samma sätt med endast små variationer. Detta kapitel bygger på information om *GNU C++ Compiler*. Denna kompilator heter `g++`. De flesta versioner av Unix har `g++`, men endal har en egen kompilator som ofta heter något i stil med `CC`, `cxx` eller dylikt. `g++` startas genom att skriva `g++` på kommandoraden. Startas den utan argument får man ett kort felmeddelande:

```
% g++  
g++: No input files  
%
```

Alldeles uppenbart skall man ge en eller flera parametrar till `g++`. Vi antar att hi har ett program i en fil som heter `Hello.cpp`. För att kompilera detta program ger vi kommandot:

```
% g++ Hello.cpp  
%
```

Om programmet inte innehåller fel eller underliga konstruktioner fås inga meddelanden överhuvudtaget från kompilatorn. Vi har nu ett program som heter `a.out`, vilket är det traditionella namnet för namnlösa program under Unix. För att ändra på det namn som kompilatorn ger åt vårt program använder vi switchen `-o` följt av ett filnamn:

```
% g++ Hello.cpp -o Hello  
%
```

Det är ingen skillnad i vilken ordning olika switchar ges åt `g++`. Det finns några fall då det är skillnad, men de nämns senare.

Optimering och debuggning

Kompilatorn kan då den genererar den färdiga programmet försöka *optimera* koden, d.v.s. göra den snabbare. Detta betyder inte att kompilatorn försöker skriva om kod som programmeraren skrivit ineffektivt, utan den försöker göra olika slingor så korta som möjligt, undvika långsam kod och använda diverse fula trick för att göra programmet snabbare. Kompilatorn kan i vissa fall försöka om och om igen göra en viss del av programmet snabbare tills den hittar en lösning som är bra. Hur mycket snabbare ett prograam kan bli beror till stor del på typen av program. Om det är ett program som till store del väntar på input från användare eller dylikt kommer en optimering av koden kanppast ens att märkas. Om det däremot är ett program som spenderar en stor del av sin tid i olika beräkningar kan optimering av koden ha en stor inverkan på exekveringstiden.

Kompilering med optimeringar innebär att kompilatorn konsumerar mycket mera minne och tid för kompileringen. Det kan därför löna sig att under utvecklingsskedet kompilera program utan optimeringar, och först då programmet är klart görs en sista kompilering med optimeringar påslagna. Ofta blir de resulterande kärbara filerna en aning större än de skulle blivit utan optimeringar, med skillnaden är oftast mycket liten. För att kompilera med optimeringar använda flaggan `-O` eller `-On`, där `n` är ett värde mellan 0 och 3 (inklusive). Värdet 0 innebär ingen optimering överhuvudtaget och 3 full optimering. För att optimera vårt program `Hello.cpp` kan vi göra följande:

```
% g++ -O2 Hello.cpp -o Hello
%
```

Vi kunde även använda endast `-O`, vilket är ekvivalen med `-O2`.

Om vi har problem med vårt program och vill veta vad det gör finns det två huvudalternativ. Det ena är arbetsdrygt och innebär att man med jämna mellanrum i programmet skriver ut data om vad programmet gör, t.ex. värden på centrala variabler. Den andra och mera praktiska metoden innebär att man använder en *debugger*. En debugger kan köra ett program under kontrollerade omständigheter och låter programmeraren titta på variablers värden, köra programmet radvis o.s.v. Det finns olika debuggers under Unix, bl.a. `gdb` (GNU Debugger) och `dbx`. På Linux används `gdb`. För att debuggern skall kunna "debugga" vårt program måste vi instruera kompilatorn att skriva med extra information i det slutgiltiga programmet. Denna extra information innehåller data om variabler, deras typ och namn, själv programkoden som den ser ut i originalfilen o.s.v. Program med debug-information inkluderar är vanligen *mycket* större än program utan samma information. Ofta är det fråga om filstorlekar på tiotals gånger normal storlek. Stora program är långsamma att starta, och ibland kör program med debuginformation även långsammare. Det lönar sig alltså att endast inkludera debug-information då den verkligen behövs. För att göra detta används switchen `-g`. Om vi igen ser på vårt exempel:

```
% g++ Hello.cpp -g -o Hello
%
```

Det är inte heller här någon skillnad vart switchar placeras, så länge de inte placeras mellan en switch och dess argument (t.ex. `-o` och `Hello` ovan).

Varningar

Ofta då man programmerar gör man små fel som egentligen inte direkt är fel, men som kan bli det med lite otur. Man skriver då kod som är tvivelaktig och som borde ändras för att den ska uppfylla standarderna för C++. I sådana fall kan man få kompilatorn att generera *varningar* för kod som är tvivelaktig. I standardutförande ger t.ex. `g++` mycket få varningar, d.v.s. den tillåter kod som är mycket nära syntaktiskt fel. Om man inte korrigerar dessa olägenheter kanske de orsakar fel senare i programmet. Till `g++` kan man ge switchen `-Wall` för att den skall generera alla möjliga varningar. Det är en mycket bra sed att alltid se till att alla program kompilerar helt utan varningar, eftersom det är ett tecken på en bra programmerare. Vi kan kompilera vårt `Hello.cpp` med varningar påkopplade, men det ger knappast några fel. Istället kan man prova kompilera nedanstående program, so innehåller två små missar:

```
#include <ostream>

int main () {
    // oanvänd variabel
    int Oanvand;

    enum Alternativ { A, B, C };
    Alternativ Tal = A;

    // switch som ej beaktar värdet 'C'
    switch ( Tal ) {
        case A : cout << "Det är ett A!" << endl; break;
        case B : cout << "Det är ett B!" << endl; break;
    }
}
```

Bara genom att läsa programmet kan det vara svårt att se vad som kunde vara fel på det. Kompilerar men det på normalt sätt får man heller inga varningar. Vi antar att programmet heter `Varning.cpp`.

```
% g++ Varning.cpp -o Varning
%
```

Inga varningar. Läger vi till `-Wall` får vi följande resultat:

```
% g++ -Wall Varning.cpp -o Varning
Varning.cpp: In function `int main()':
Varning.cpp:13: warning: enumeration value `C' not handled in switch
Varning.cpp:5: warning: unused variable `int Oanvand'
%
```

Nu kanske vi ser vad det är frågan om. Vi har en variabel `Oanvand` som vi nog deklarerar, men aldrig använder. Det är orsak till en varning. Vi har även en `switch`-konstruktion som kontrollerar en `enum`. Vi har dock ingen `case` för värdet `C`, och inte har vi heller en `default` som kunde "fånga upp" alla som inte har en egen `case`. Även detta är värt en varning.

Förutom `-Wall` kan man för `g++` även använda switchen `-pedantic`, som gör att kompilatorn är pedantisk då den kontrollerar koden. ger ännu mera potentiella varningar än `-Wall`. Man kan med fördel kombinera dessa två switchar och använda dem samtidigt.

Sökstig för headerfiler

Då man använder sig av externa bibliotek behöver man för det mesta inkludera speciella header-filer. I många fall är dessa header-filer placerade på ett ställe på disken där kompilatorn söker automatiskt, t.ex. `/usr/include`, men alltid är de inte det. I sådana fall måste man kunna berätta åt kompilatorn var den skall söka efter header-filer som inte hittas i normala kataloger. Switchen som gör detta är `-I`*katalog*. Om vi har ett program som heter `StortProgram.cpp` som behöver header-filer som finns i både `/opt/Mesa/include` och `/usr/local/include` kan man ge följande kommandorad:

```
% g++ StortProgram.cpp -I/usr/local/include -I/opt/Mesa/include
%
```

Man kan alltså ge flera `-I` på samma kommandorad. De läggs sedan till till den interna sökvägen för kompilatorn. Notera att inget mellanrum finns mellan `-I` och katalognamnet. Om alla header-filer som `StortProgram.cpp` behöver inte kan hittas ens i dessa kataloger ger kompilatorn ett felmeddelande. Man måste då lägga till ytterligare sökstigar med nya `-I`.

[Förutgående](#)

Minneshantering på C:s vis

[Hem](#)

[Nästa](#)

Länkning av program